

搜索结构

• 搜索：在数据集中寻找满足某种条件的数据对象。结果分为成功和失败。

搜索结构：用于搜索的数据结构，通常每个对象有若干属性，搜索即用其“关键词”

* 静态搜索表：数据集在搜索过程中不发生变化。

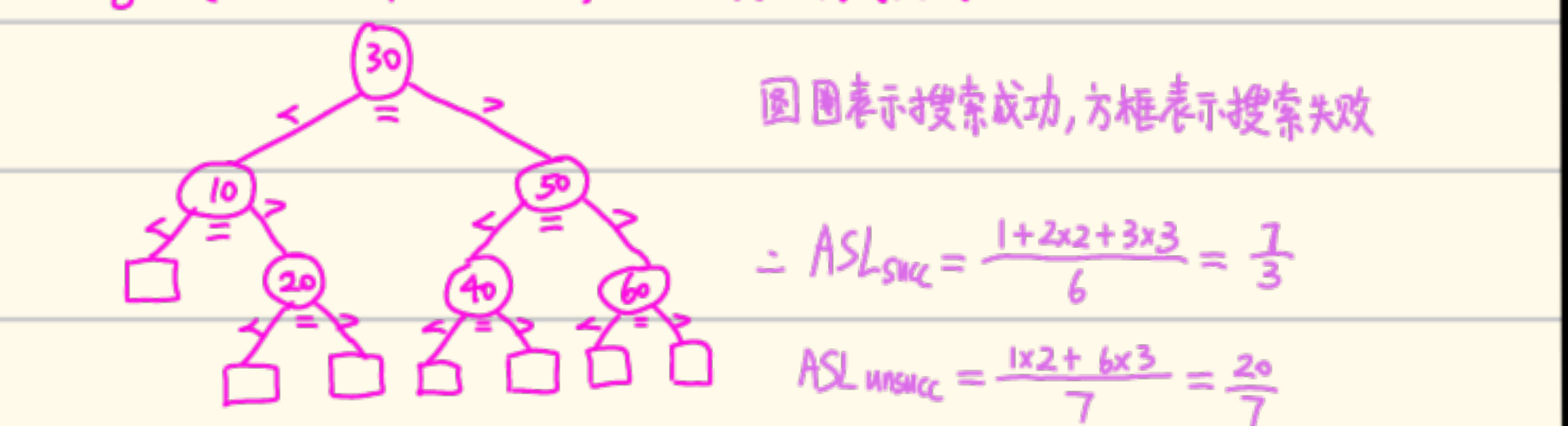
• 顺序搜索：在线性表中，从前往后比较，找到则搜索成功；遍历完仍未找到则搜索失败

$$ASL_{succ} = \sum_{i=1}^n p_i \cdot i, \text{ 等概率模型即 } \frac{n+1}{2}$$

$$ASL_{unsucc} = n+1 \quad (\text{效率较低})$$

• 折半搜索：查找表必须有序，每次比较中间元素并缩小一半搜索范围。

eg: (10, 20, 30, 40, 50, 60) 折半搜索的判定树：



(效率高，但因为是有序表，插入、删除均困难)

* 二叉搜索树：(动态搜索结构，可高效查找、插入、删除)

• 定义：若树非空，则左子树上的所有结点关键词均小于根结点；右子树上的所有结点

关键词均大于根结点；左、右子树也是二叉搜索树。

* 二叉搜索树中序遍历得到的序列即为从小到大排好序的。

• 搜索思路：给定根结点，若为NULL则搜索失败，否则若给定值小于root->key，则

Search (root->leftchild)；若等于则成功；若大于则 Search (root->rightchild)

(最多比较次数为树高h)

• 插入思路：给待插结点，记录其key值，然后按搜索的思路从根开始递归地比，

若比到相等的则直接返回(二叉搜索树不允许相等元素)；否则在最后搜到的NULL

位置上放待插结点即可。(建立二叉搜索树即插n-1次)

• 删除思路：分类讨论：找到待删结点后，若其为叶结点，则直接删；

若其左子树为空，则用其右子女结点顶替；若其右子树为空，则用其左子女结点顶替；

若左右均不空，则在其右子树下寻找中序下的第1个结点，用其顶替被删结点后再递归处理

这个结点的删除问题。

* AVL树：左右子树高度之差的绝对值不超过1的二叉搜索树

每个结点附加一个数字bf(平衡因子)，即该结点的右子树高度减去左子树高度

显然，AVL树任一结点的bf只能是-1, 0, 1；这样保证单次搜索和ASL均为log级别

• 通过旋转操作来保持平衡：

(1) 左单旋转：



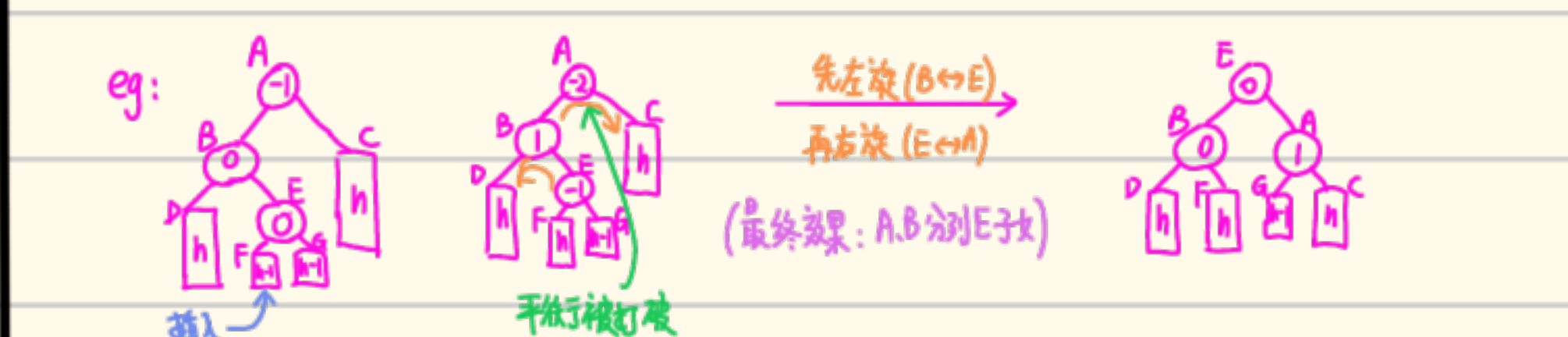
操作：① 传入当前根ptr=A，记temp=ptr ② ptr=ptr->right (即C)

③ temp->right=ptr->left (即D改接A) ④ ptr->left=temp (即A改做C左子)

⑤. ptr->bf=temp->bf=0 (重赋平衡因子) ⑥ 返回新根ptr=C

(2) 右单旋转：同“左单旋转”，将所有左右互换即可。

(2) 先左后右双旋转：即先左旋，再右旋。



(4) 先右后左双旋转：同“先左后右双旋转”，将所有左右互换即可。

• AVL树的插入

按二叉搜索树找到待插入位置，插入后进行旋转来保持操作。

具体方法：插入后，新节点P的bf=0，若其为父节点Pr的右子女，则Pr.bf++，否则Pr.bf--。

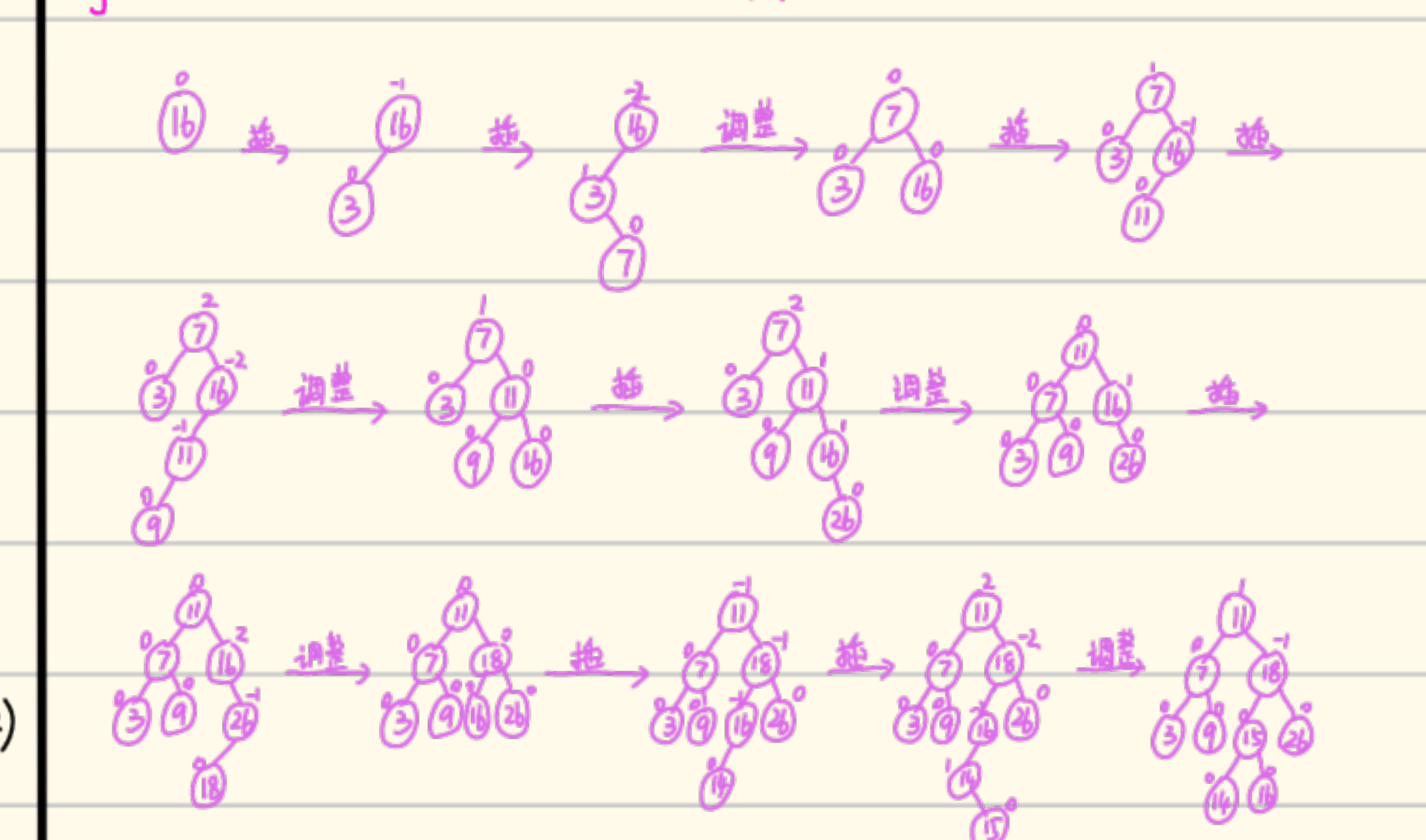
若Pr.bf=0：说明恰平衡，直接返回

若|Pr.bf|=1：说明Pr这棵树没有失衡，但需沿Pr到根的方向回溯，逐一检查平衡状态

若|Pr.bf|=2：说明造成了不平衡，根据Pr.bf的正负和Pr的孩子的bf的正负来判断：

bf 0 则左单；bf 0 则右左双；bf 0 则右单；bf 0 则左双。调整后无需向上回溯检查

eg: 关键词序列为{16, 3, 7, 11, 9, 26, 18, 14, 15}，建立AVL树的插入和调整过程：



• AVL树的删除：

若被删节点P有2个子女，则搜索P在中序下的直接前驱/后继q，用q替代P，随后考虑q的删

除问题；若被删节点P最多只有1个子女q，则将P的父节点中原来指向P的指针改指q，然

后Pr为根的子树高度减一，故同插入时向上处理(遇不平衡就旋转)即可。

• AVL树性能分析：

有n个结点的AVL树的高度不超过1.44 * log2(n+2)

AVL树中，搜索、插入、删除一个结点并做平衡化旋转所需时间为O(logn)